Greatest challenges that prevent business from capitalizing on big data

- Obtaining executive sponserships for investments in big data and its related activities(such as training etc)
- Getting the business units to share information across organizational tower.
- Finding the right skills (business analysts and data scientists)
- Determining the approach to scale rapidly and elastically
- Deciding whether to use structured or unstructured, internal or external data to make business decisions
- Choosing the optimal way to report findings and analysis of big data for the prsentations to make more sense
- Determining what to do with the insights created from big data

Top challenges facing Big data

- **Scale:** Should you scale vertically or horizontally ?
- **Security:** Most of big data platforms have poor security mechanisms
- **Schema:** Rigid schemas have no place. The need of the hour is dynamic schema
- **Continuous Availability:** How to provide 24/7 support
- **Consistency:** Should one opt for consistency or eventual consistency?
- **Partition tolerant:** How to build partition tolerant systems that can take care of both hardware and software failures?
- **Data quality:** how to maintain accuracy, completeness, timeliness etc

Termnologies used in Big Data Environments

1. In-Memory Analytics:

- In-memory analytics is a business intelligence (BI) methodology used to solve complex and time-sensitive business scenarios.
- It works by increasing the speed, performance and reliability when querying data.
- Business Intelligence deployments are typically disk-based, meaning the application queries data stored on physical disks. In contrast, with in-memory analytics, the queries and data reside in the server's random access memory (RAM)
- In-memory analytics helps improve the overall speed of a BI system and provides business-intelligence users with faster answers compared to traditional disk-based business intelligence, especially for queries that take a long time to process in a large database.

2. In-database processing:

- In-database processing, sometimes referred to as in-database analytics, refers to the integration of data analytics into data warehousing functionality.
- In-database analytics is a technology that allows data processing to be conducted within the database by building analytic logic into the database itself.
- Doing so eliminates the time and effort required to transform data and move it back and forth between a database and a separate analytics application.

Data doesn't have to be extracted, transformed, and reloaded (ETL) for analysis.

Everything happens where the data already resides → faster and more efficient

This technology is widely adopted in large databases, including those applied in credit card fraud detection

3. Symmetric Multiprocessor System (SMP):

Symmetric multiprocessing (SMP) is a **multiprocessor computer architecture** where:

- Two or more **identical processors** are connected to a **single shared memory**.
- All processors have equal access to **I/O devices** and system resources.
- A **single operating system** controls and treats all processors equally (none has higher priority).
- This is the most common architecture used in **modern multiprocessor systems**, including **multi-core CPUs** (where each core is treated like a separate processor).

•**Processors (P1, P2, … Pn)** → Multiple identical CPUs that execute tasks.

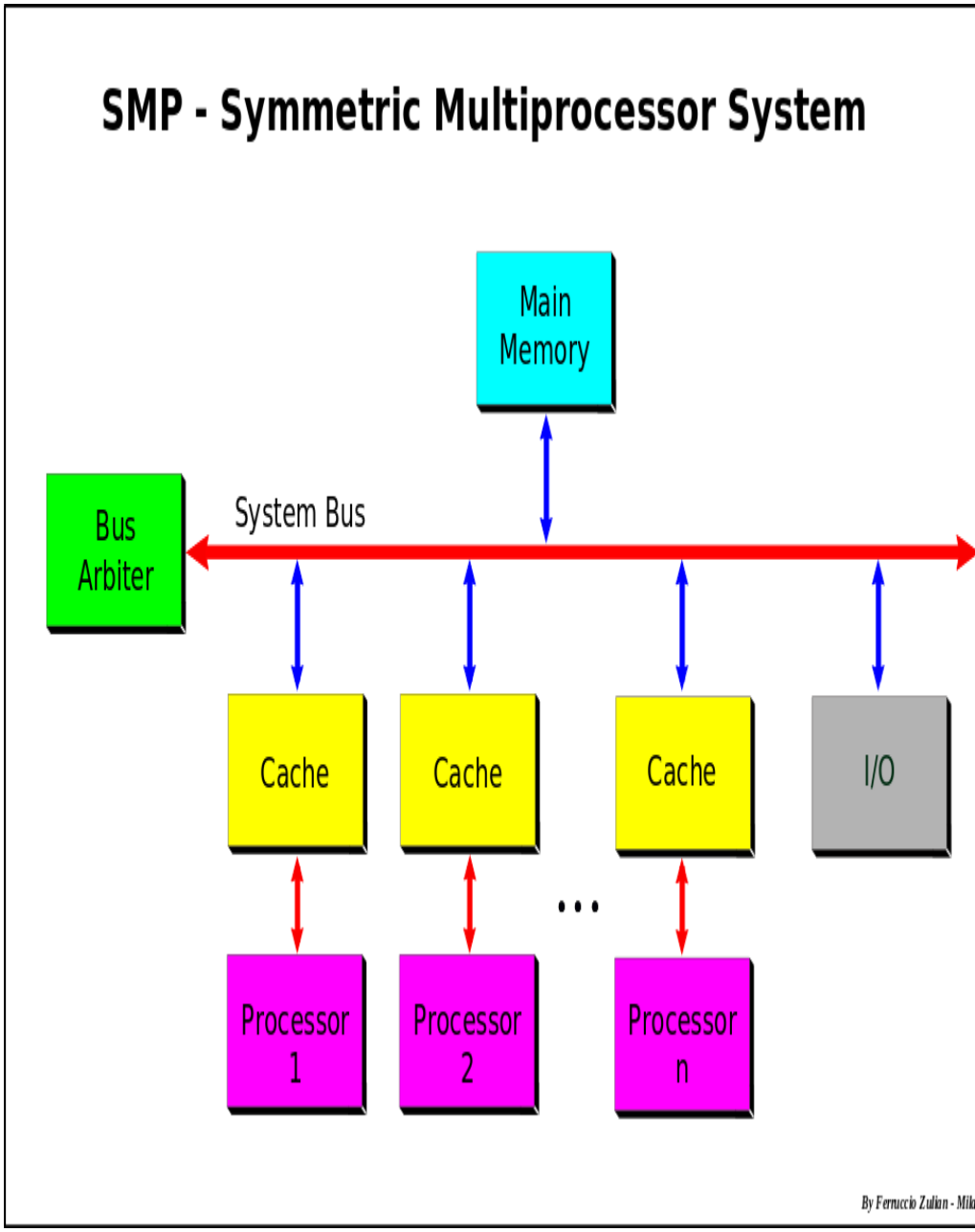
•**Cache** → Each processor has its own cache to store frequently used data.

•**System Bus** → A common pathway that connects all processors, memory, and I/O.

•**Main Memory** → Shared memory accessible to all processors.

•**I/O devices** → Shared input/output resources accessible by all CPUs.

•**Bus Arbiter** → Manages access to the system bus, ensuring no two processors use it simultaneously.



•Each processor executes its own program independently.
•They can all access **shared memory** and **I/O devices** via the system bus.
•The **bus arbiter** prevents conflicts when multiple processors attempt to access the bus at the same time.
•Since processors are identical, the **operating system can schedule any process on any CPU**, improving performance.

Characteristics:
Tightly coupled system – All processors are connected and work closely together.
Homogeneous processors – All CPUs are identical and capable of performing any task.
Shared resources – Memory, I/O devices, and interrupt systems are common to all processors.
System Bus / Crossbar – Communication between processors and memory/I/O is done through a shared bus or switching mechanism.

4. Massively Parallel Processing (MPP):

•**MPP (Massively Parallel Processing)** means breaking a big program into smaller parts and running them **in parallel** on **many processors**.
•Each processor has its **own memory, cache, I/O system, and even its own operating system**.
•The processors (nodes) don't share memory directly — they communicate using an **interconnection network** and a **messaging interface**.
•This is why MPP is called a **"shared-nothing architecture"** (unlike SMP where memory and I/O are shared).

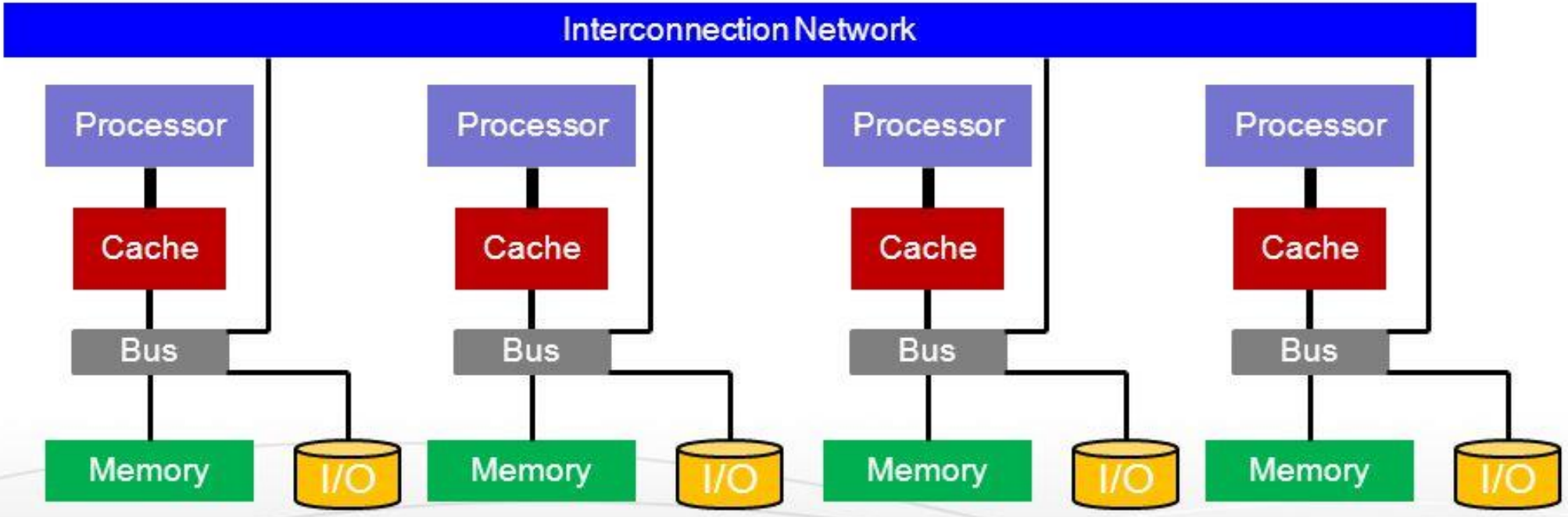
•**Independent Processors** → Each has its own CPU + Cache.
•**Private Memory & I/O** → No processor shares memory or storage with another.
•**Interconnection Network** → A communication link between processors.
•**Independent OS** → Each node can run its own operating system instance.
•**Loosely Coupled** → Each processor works independently, only communicating when necessary.

Data Warehouses
Examples: **Amazon Redshift, Teradata, Google BigQuery**.
When you run a query on billions of records:

Database is split across multiple processors.
Each processor works on a portion of the data stored in its memory.
Results are combined and returned very quickly.

Massively Parallel Processors

- Massively Parallel Processors (MPP) architecture consists of nodes with each having its own processor, memory and I/O subsystem



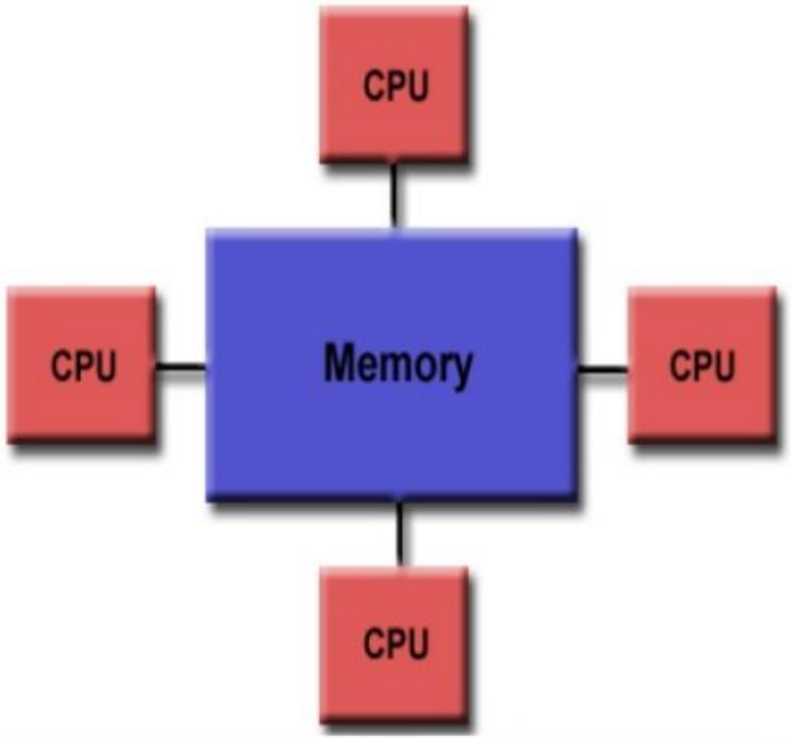
- An independent OS runs at each node

5. Difference between Parellel and Distributed systems

Parallel system:

- A system is said to be a **Parallel System** in which multiple processor have direct access to shared memory which forms a common address space.
- Usually tightly-coupled system are referred to as **Parallel System**. In these systems, there is a single system wide primary memory (address space) that is shared by all the processors. On the other hand **Distributed System** are loosely-coupled system.
- **Parallel computing** is the use of two or more processors (cores, computers) in combination to solve a single problem.

A Parallel System



Advantages of Parallel System

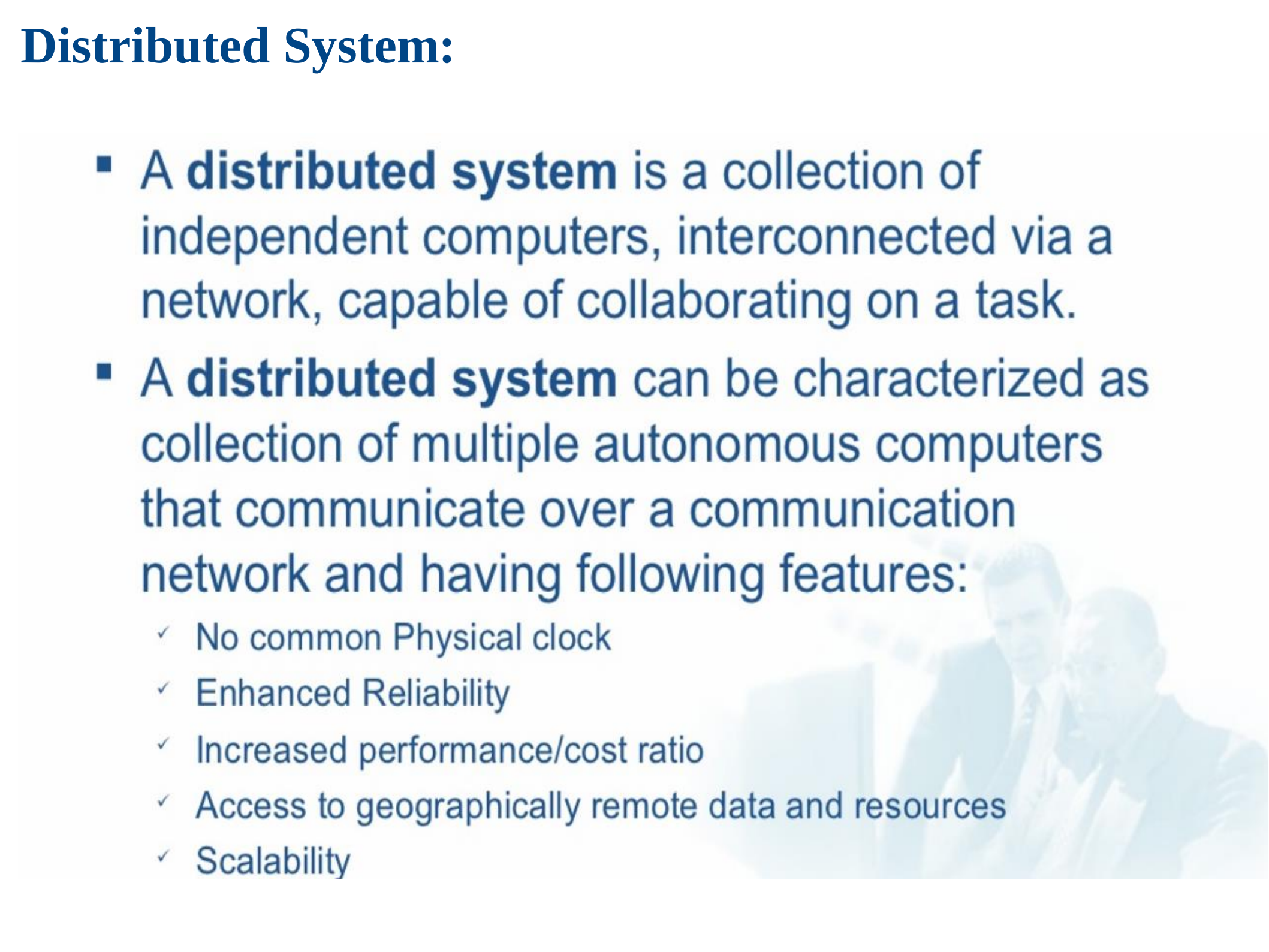
- Provide Concurrency(do multiple things at the same time)
- Taking advantage of non-local resources
- Cost Savings
- Overcoming memory constraints
- Save time and money
- Global address space provides a user-friendly programming perspective to memory

Disadvantages of Parallel System

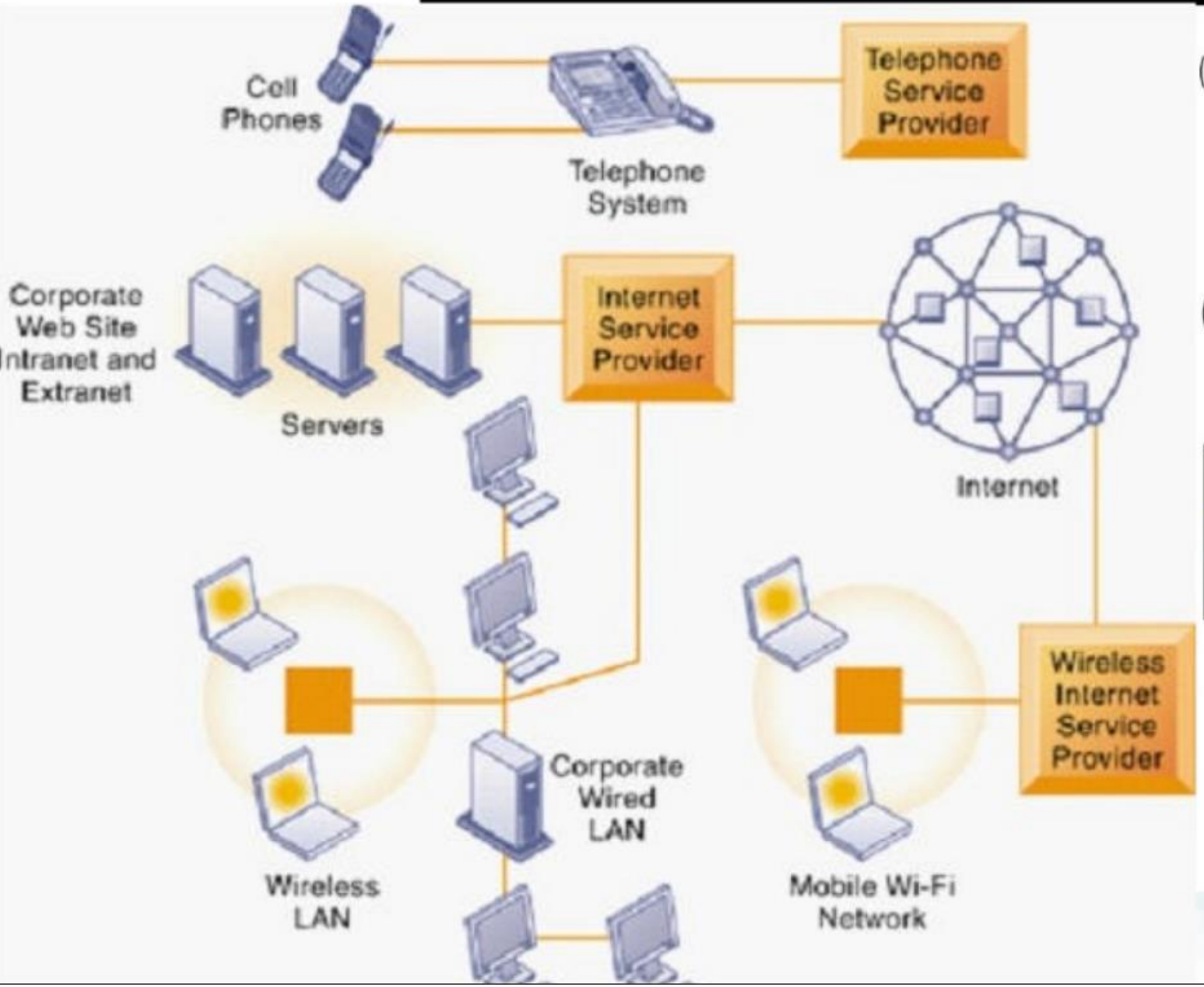
- Primary disadvantage is the lack of scalability between memory and CPUs.
- Programmer responsibility for synchronization constructs that ensure "correct" access of global memory.
- It becomes increasingly difficult and expensive to design and produce shared memory machines with ever increasing numbers of processors.

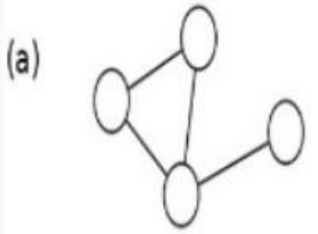
Distributed System:

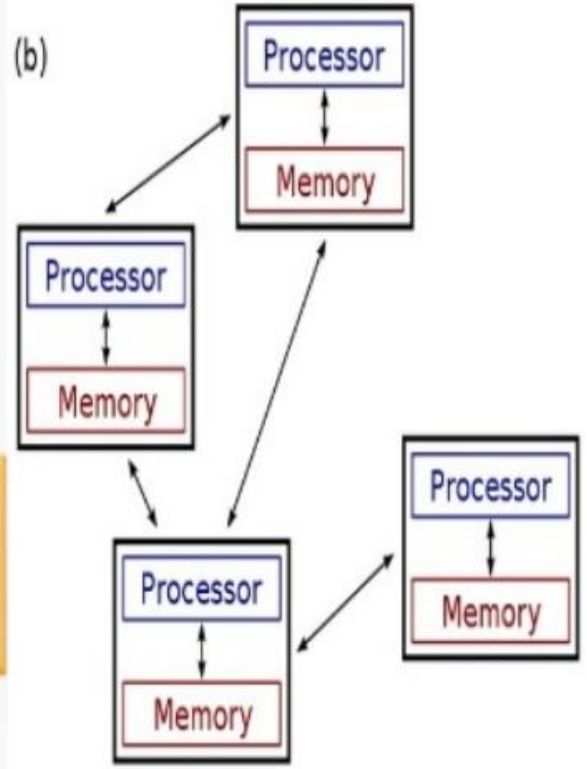
- A **distributed system** is a collection of independent computers, interconnected via a network, capable of collaborating on a task.
- A **distributed system** can be characterized as collection of multiple autonomous computers that communicate over a communication network and having following features:
  - ✓ No common Physical clock
  - ✓ Enhanced Reliability
  - ✓ Increased performance/cost ratio
  - ✓ Access to geographically remote data and resources
  - ✓ Scalability



A Distributed System





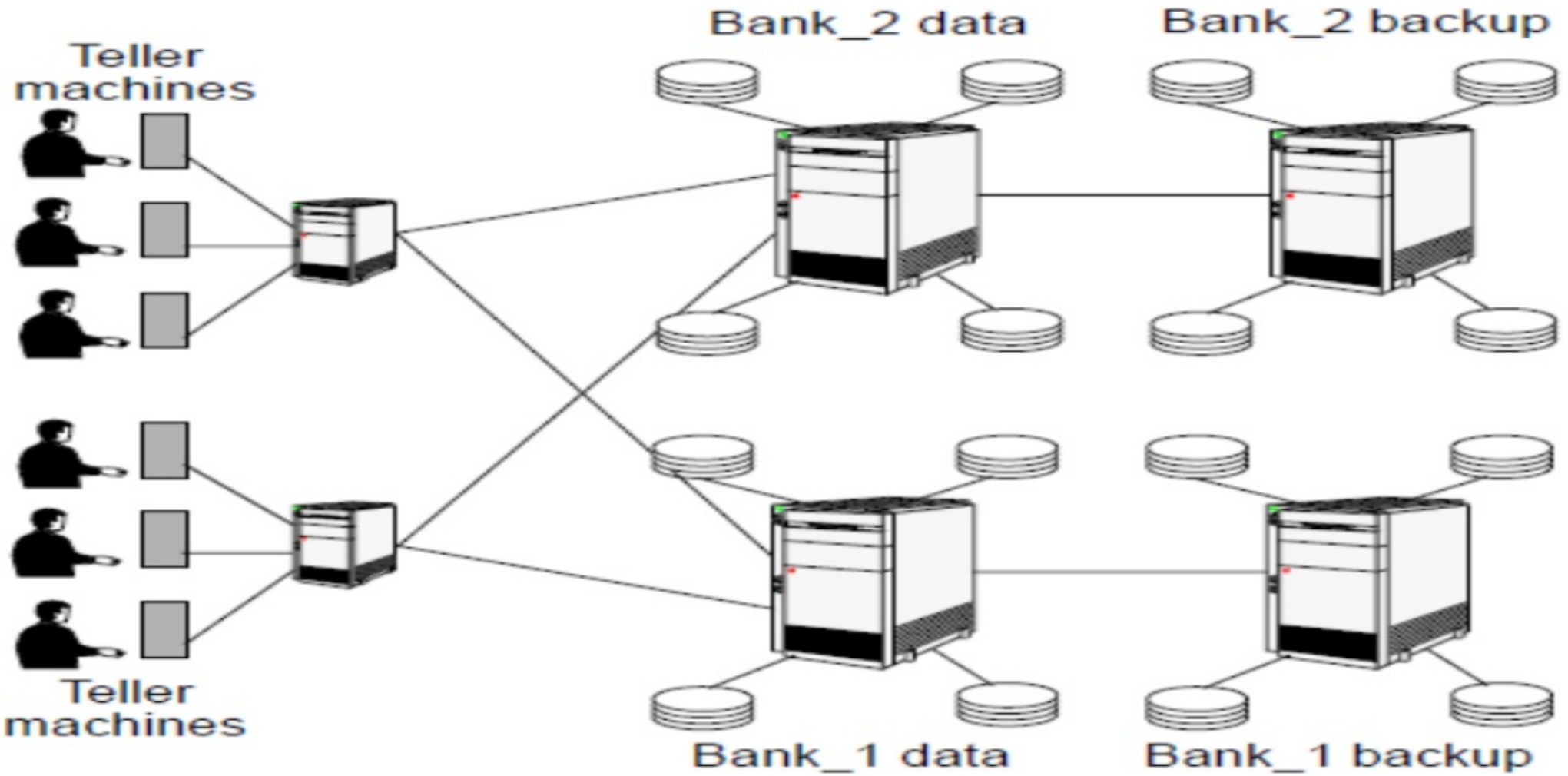


Examples of Distributed System

- Telephone Networks and Cellular Networks
- Computer Networks Such as internet or intranet
- ATM(bank) Machines
- Distributed database and distributed database management system
- Network of Workstations
- Mobile Computing etc.

Examples of Distributed Systems (cont'd)

Automatic banking (teller machine) system



- Primary requirements: security and reliability.
- Consistency of replicated data.
- Concurrent transactions (operations which involve accounts in different banks; simultaneous access from several users, etc).

Advantages Of Distributed System

- Information Sharing among Distributed Users
- Resource Sharing
- Extensibility and Incremental growth
- Shorter Response Time and Higher Output
- Higher Reliability
- Better Flexibility's in meeting User's needs
- Better price/performance ratio
- Scalability
- Transparency

Disadvantages of Distributed System

- Difficulties of developing distributed software
- Networking Problem
- Security Problems
- Performance
- Openness
- Reliability and Fault Tolerance

Parallel vs. Distributed System

	Parallel Systems	Distributed Systems
Memory	Tightly coupled system shared memory	Weakly coupled system Distributed memory
Control	Global clock control	No global clock control
Processor interconnection	Order of Tbps	Order of Gbps
Main focus	Performance Scientific computing	Performance(cost and scalability) Reliability/availability Information/resource sharing

6. Shared Nothing architecture

➢Three most common types of architecture for multiprocessor high transaction rate systems are:

➢**a. Shared Memory (SM)**

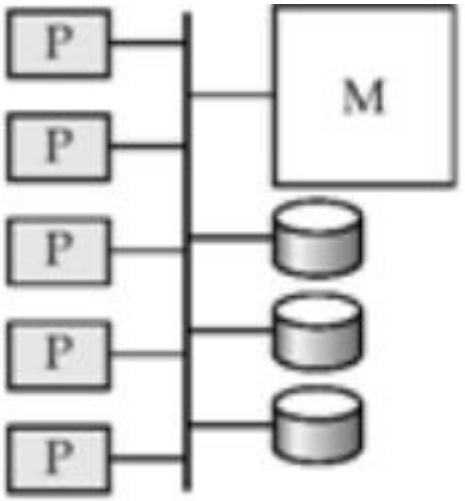
--- A common central memory is shared my multiple processors.

➢**b. Shared Disk (SD)**

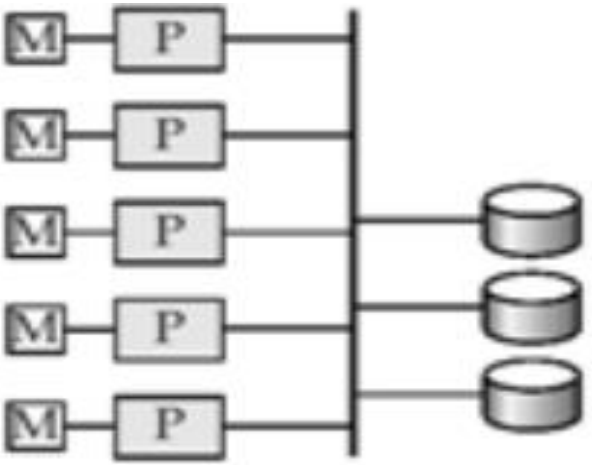
--- Multiple processors have a common collection of disks having their own private memory.

➢c. **Shared Nothing (SN)**

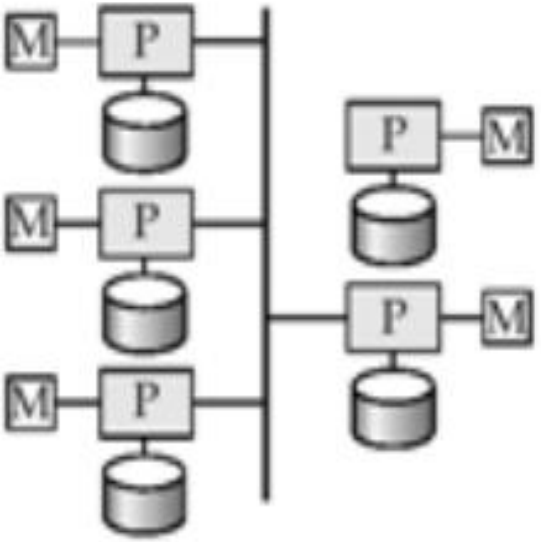
--- Neither memory, nor disk is shared among the multiple processors.



(a) shared memory



(b) shared disk



(c) shared nothing

Advantages and Disadvantages of SN Architecture

Advantages

- **Elimination of SPOF (single point of failure)** – if a part of a system fails, it won't stop the entire system from working
- **Non-disruptive upgrades** – database updates will not interrupt access to database data
- **No need for reboots** – SN does not require system as a whole to be rebooted when the update process completes.
- **Self-healing capabilities** – allows a database to be repaired while still processing transactions

Disadvantages

- Hard to program
- Addition of new nodes requires reorganizing

7. CAP Theorem

It is also called as ***Brewer's Theorem*** or ***two out of three theorem***.

" It states that in a distributed computing environment, it is impossible to provide the following guanrantees. At the best you can have two of the following three – one must be sacrified."

1. Consistency

2. Availability

3. Partition Tolerance

Example: Updating a status in a Facebook.

We do not want any type of consistency and also we do not want that our whole servers are available because it may be happens that it is available to some people.

Consistency –This means that the data in the database remains consistent after the execution of an operation. For example after an update operation all clients see the same data.

Availability – This means that the system is always on (service guarantee availability), no downtime. ***Partition Tolerance*** – This means that the system continues to function even the communication among the servers is unreliable, i.e. the servers may be partitioned into multiple groups that cannot communicate with one another.

- Theoretically it is impossible to fulfill all 3 requirements.
- CAP provides the basic requirements for a distributed system to follow 2 of the 3 requirements.
- Therefore all the current NoSQL database follow the different combinations of the C, A, P from
- the CAP theorem.

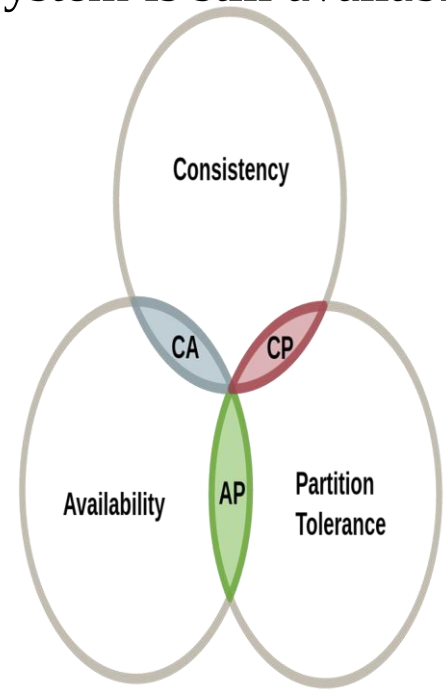
Here is the brief description of three combinations CA, CP, AP :

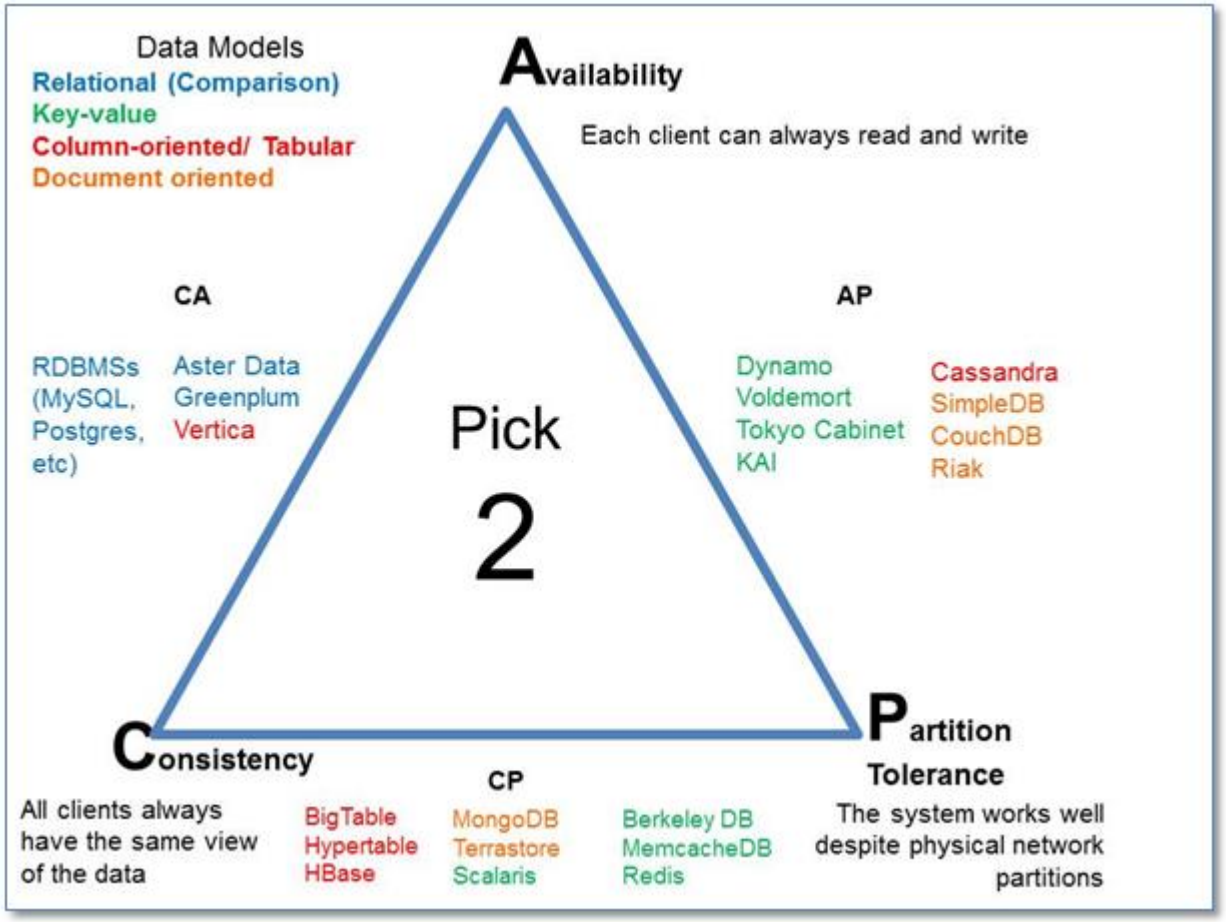
CA – Single site cluster, therefore all nodes are always in contact. When a partition occurs,

the system blocks.

CP – Some data may not be accessible, but the rest is still consistent/accurate.

AP – System is still available under pa





Summary of CAP Theorem with Case study

One can atmost go with two of the three.

1. Consistency:

→The instructors or training coordinator, once they have updated information with you,

will always get the updated information when they call subsequently.

2. Availability

→The instructors or the training coordinators will always get the schedule if any or both of the

office administrators have reported to work.

3. Partition Tolerance

→Work will go on as usual even if there is communication loss between the office administrators

→owing to a spat (past) or a tiff(quarrel)

BASE PROPERTIES

- BASE stands for Basically Available, Soft state and Eventually consistent.
- Based upon CAP theorem, NoSQL has BASE properties.
- Based upon that BASE properties it is works in a manner that it may be eventually consistent having soft state when it is basically available.
- **Basically Available:**
- When data is stored at one site, If that site is down, our data is totally unavailable but if our data is distributed to many sites, means some part of our data is at one site and some part of data is at other site.
- If one or two sites down our whole data is not unavailable.
- It may be that some part of data becomes unavailable for some short time.

Soft State:

Consistency is hard requirement of ACID properties but BASE allows that our data may be inconsistent.

It allows dynamic designing to application developers.

Eventually Consistent:

NoSQL systems assume that they provided consistency in the future, not immediately at the end of operation.

In contrast to ACID systems that enforce consistency after transaction commit,

NoSQL guarantees consistency only at some undefined future time.

So eventually consistency means it is consistent after some time even at that time it is not

ACID vs BASE PROPERTIES

ACID	BASE
It stands for Atomicity, Consistency, Isolation and Durability	It stands for Basically Available, Soft state and Eventually consistent
These are pessimistic that means it guarantees that consistency should be achieved at the end of any operation.	These are optimistic means that it provides no guarantees that consistency should be achieved at the end of any operation.
Basic requirement in RDBMS	Basic requirement in NoSQL
These are complex to implement all properties	These are simple to implement all properties
Provides reliability	Provides no reliability

When to choose consistency over availability and vice-versa....

1. Choose Availability over consistency when your business requirements allow some flexibilit around when the data in the system synchronizes.

2. Choose Consistency over Availability when your business requirements demand atomic reads and writes.